

# Option C Task Report

COS30018 – Intelligent Systems

Lecturer: Dr. Nguyen Manh Toan

Group: 3.1 – OpenBuffet

Task name: **C.4 – Machine Learning 1**



In this week task – C4: Machine Learning 1, we've done our research on how to integrate different Deep Learning Networks (LSTM, GRU and RNN) using **Keras' Sequential Model**.

We've also refactored our code to separate the concerns and make the code base maintainable.

Here are the detailed code descriptions of our works:

### 1. Task\_4/model\_builder.py/build\_model()

```
def build_model(
    sequence_length: int,
    layer_sizes: list[int],
    layer_type: str = 'LSTM',
    dropout_rate: float = 0.2,
    dense_units: int = 1,
    optimizer: str = 'adam',
    loss: str = 'mean_squared_error'
):
```

This is the required Task C.4 function. Instead of hardcoding layers manually, all architectural decisions are passed as parameters. **layer\_sizes** serves double duty - its length determines the number of layers, and each value sets the units in that layer. This means changing from a 3-layer to a 1-layer network only requires changing the list, not rewriting model code.

```
layer_type_upper = layer_type.strip().upper()
layer_registry = {
    'LSTM': LSTM,
    'GRU': GRU,
    'RNN': SimpleRNN,
}
if layer_type_upper not in layer_registry:
    raise ValueError(
        f"Unknown layer_type '{layer_type}'. "
        f"Choose from: {list(layer_registry.keys())}"
    )
RecurrentLayer = layer_registry[layer_type_upper]
```

Rather than using if/elif chains to pick the layer class, a dictionary maps string names to Keras layer classes. **RecurrentLayer** then holds the actual class object (not an instance), so it can be called like **RecurrentLayer(units=128)** later in the loop. This is the Python pattern of treating functions/classes as first-class objects.

```

for i, units in enumerate(layer_sizes):
    # All layers except the last must return their full output sequence so
    # the next recurrent layer receives a 3-D tensor (batch, timesteps, units)
    # instead of just the last hidden state (batch, units).
    is_last_recurrent = (i == n_layers - 1)
    return_sequences = not is_last_recurrent

    model.add(RecurrentLayer(units=units, return_sequences=return_sequences))

    # Dropout after every recurrent layer (skip when rate == 0).
    if dropout_rate > 0.0:
        model.add(Dropout(rate=dropout_rate))

```

In this code, **return\_sequences=True** tells a recurrent layer to output its hidden state at **every timestep**, producing a 3D tensor (batch, timesteps, units). **return\_sequences=False** (the default) outputs only the **final** hidden state, collapsing to 2D (batch, units). Every layer except the last must use **return\_sequences=True** and if you set it to False on an intermediate layer, the next recurrent layer receives a 2D tensor and crashes because it expects a sequence to iterate over [1].

Next is “**Dropout**”, it randomly sets a fraction (**dropout\_rate**) of neuron outputs to zero during each training step. This prevents neurons from co-adapting - where one neuron compensates for the mistakes of another - forcing the network to learn more robust, distributed representations.

```

model.add(Dense(units=dense_units))

```

A **Dense layer** with no activation function (linear by default) maps the final hidden state to the predicted price. No activation is correct here because this is a **regression** task - we want an unbounded real number output, not a probability or a value squashed into [0, 1]. Using sigmoid or relu here would artificially constrain the prediction range.

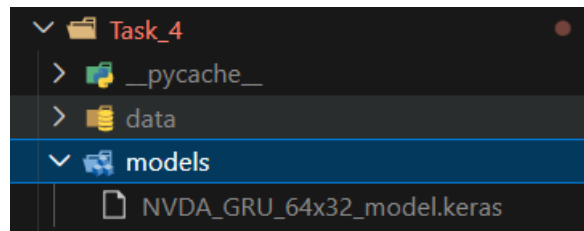
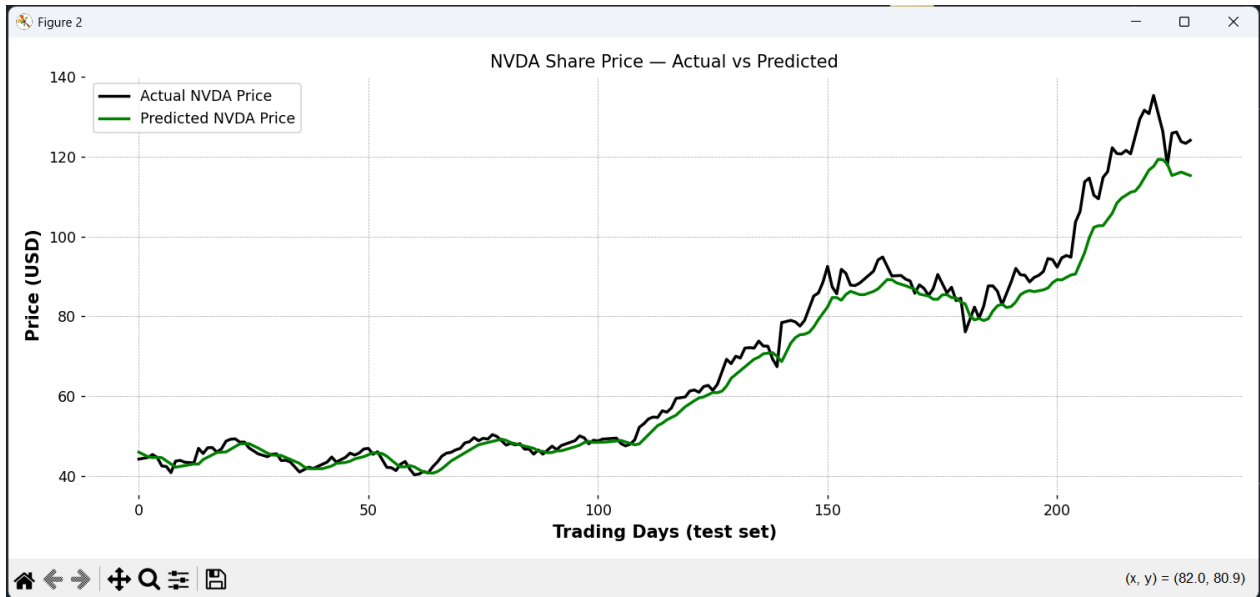
```

model.compile(optimizer=optimizer, loss=loss)

```

Finally is the `compile()`, **adam (Adaptive Moment Estimation)** is used as the optimiser. It combines momentum (using past gradients to smooth updates) and adaptive learning rates (scaling updates per parameter), making it converge reliably without manual learning rate tuning. `mean_squared_error` is the loss function standard for regression - it penalises large errors more heavily than small ones due to the squaring, which suits stock price prediction where large mispredictions are costly.

I ran a GRU network, the model then saved in `models/`



## 2. Code Base Refactoring

Our team has decided to moving from the original monolithic approach where everything being held in a same Python file to modularize weekly tasks.

**data\_loader.py** ← Task C.2: download, NaN handling, split, scale

**visualiser.py** ← Task C.3: candlestick, boxplot, prediction plot, CLI helpers

**model\_builder.py** ← Task C.4: build\_model() + save/load helpers

**predictor.py** ← test-set windowing, inference, next-day forecast